Projektova zprava
Nejlepsi tamagotchi na celym sigma svete

Matyas Potocký a Martin Veber

27. 4. 2026

Obsah

1 Zakladni informace **2**

2 Cil aplikace **2**

3 Pouzite technologie **2**

4 Architektura a struktura komponent **3**

5 Sprava stavu **6**

6 Animace a vzhled **7**

7 Nasazeni **7**

8 PWA **8**

9 Testovani **8**

10 Rozdeleni prace **12**

11 Zhodnoceni projektu **12**

12 Prilohy **13**

1 Zakladni informace

Nazev projektu: Nejlepsi tamagotchi na celym sigma svete

Jednoduche shrnuti: Jde o webovou hru, ve ktere si uzivatel vybere virtualniho mazlicka a potom se o nej stara. Mazlicek postupne ztraci energii, zdravi, hlad i stesti, hrac proto musi provadet akce ve spravnou chvili a hlidat, aby zadny ukazatel neklesl na nulu.

Clenove tymu:

- Role A – UI: Martin Veber. Mel na starosti wireframe, mobilni rozlozeni, vizualni styl, obrazovky, animace mistnosti a finalni vzhled aplikace.
- Role B – logika: Matyas Potocký. Mel na starosti datove modely, stav aplikace, herni smycku, ukladani, CI, nasazeni a podporne PWA assety.

Repozitar: https://github.com/cqeta1564/nejlepsi-tamagotchi-na-celym-sigma-svete

Nasazena aplikace: https://cqeta1564.github.io/nejlepsi-tamagotchi-na-celym-sigma-svet

2 Cil aplikace

Cilem aplikace je vytvorit jednoduchou a srozumitelnou hru, ktera pripomina klasicke Tamagotchi. Uzivatel si vybere jednoho virtualniho mazlicka a musi se o nej starat tak, aby mu neklesly dulezite potreby na nulu. Aplikace je urcena hlavne pro studenty nebo bezne uzivatele, kteri si chteji rychle vyzkouset kratkou browserovou hru bez instalace a bez sloziteho ovladani.

Problem, ktery aplikace resi, neni vazny realny problem, ale herni situace: hrac chce mit prehlednou, rychlou a zabavnou peci o virtualniho mazlicka v jednom okne prohlizece. Aplikace proto uzivateli ukazuje jen to nejdulezitejsi: koho ma vybraneho, v jake mistnosti se prave nachazi, kolik ma penez a jak na tom mazlicek je. Vsechno je navrzene tak, aby tomu rozumel i nekdo, kdo neprogramuje.

3 Pouzite technologie

- React 19 a React DOM 19
- TypeScript 5.9
- Vite 8
- CSS v souboru `src/index.css`
- Tailwind CSS 4 jako soucast PostCSS build pipeline
- PostCSS a Autoprefixer
- ESLint 9, `typescript-eslint`, `eslint-plugin-react-hooks`, `eslint-plugin-react-refresh`
- Node.js test runner pro zakladni jednotkove testy
- Canvas API pro orezavani obrazku mazlicku a ikon v `src/utils/imageCutout.ts`
- Node.js 24 v GitHub Actions workflow

- Git a GitHub
- GitHub Actions pro CI a deploy
- GitHub Pages pro hosting produkcni verze
- `localStorage` v prohlizeci pro obnovu rozehrane hry a ulozeni motivu
- PWA vrstva: `manifest.webmanifest`, ikony, registrace service workeru a generovany `sw.js`
- AI pomocnici pri vyvoji: pouziti GitHub Copilot/Codex asistovanych uprav

4 Architektura a struktura komponent

Rozdeleni aplikace

Aplikace je rozdelena na hlavni kontejner `App`, dve hlavni obrazovky a vice mensich prezentacnich komponent. Vetsina dat a logiky vznika v `App.tsx`. Podrizene komponenty dostavaji data pres props a posilaji udalosti zpatky nahoru pres callback funkce.

Aktualni zdrojove slozky jsou:

- `src/components` – znovupouzitelne UI komponenty,
- `src/screens` – slozene obrazovky vyberu a hry,
- `src/data` – ukazkova data mazlicku,
- `src/game` – cista herni logika a pravidla akci v mistnostech,
- `src/utils` – pomocne funkce pro obrazky,
- `src/assets` – obrazky mazlicku, mistnosti a ikon.

Aktualni strom aktivnich komponent

- `App`
- `PetSelectionScreen`
- `PetSelectionList`
- `PetOptionCard`
- `PetAvatar`
- `CutoutImage`
- `SelectPetButton`
- `HomeScreen`
- `StatsPanel`
- `StatChip` (vnitrni pomocna komponenta uvnitr `StatsPanel`)
- `PetCard`

- `PetFace`
- `StatusModal`

Starsi prototypove komponenty, ktere uz nebyly soucasti render stromu, byly z kodu odstraneny. Aktualni render strom tak odpovida skutecne pouzivanym obrazovkam a komponentam.

Komponenty a jejich props

`App` Hlavni korenova komponenta. Rozhoduje, jestli se zobrazi vyber mazlicka, herni obrazovka, nacitani nebo systemova chyba. Props nema. Drzi herni data, motiv, intervaly, uklada stav do `localStorage` a obsluhuje vsechny dulezite udalosti.

`PetSelectionScreen`
Obrazovka pro vyber mazlicka. Zobrazuje seznam mazlicku a tlacitko pro potvrzeni vyberu. Prijima props `pets`, `selectedPetId`, `onSelectPet`, `onConfirmSelection`.

`PetSelectionList`
Kontejner, ktery vyrenderuje karty vsech dostupnych mazlicku. Prijima props `pets`, `selectedPetId`, `onSelect`.

`PetOptionCard`
Jedna karta ve vyberu mazlicka. Zobrazuje avatar a jmeno. Prijima props `pet`, `isSelected`, `onSelect`. Po kliknuti vola callback s id mazlicka.

`SelectPetButton`
Tlacitko pro potvrzeni vyberu. Prijima props `disabled` a `onClick`.

`HomeScreen`
Hlavni herni obrazovka po startu hry. Sklada panel statistik a hlavni kartu mistnosti. Prijima props `pet`, `roomId`, `roomIndex`, `roomCount`, `roomName`, `roomDescription`, `roomBackgroundImage`, `actionIcon`, `actionLabel`, `actionCost`, `coins`, `statusText`, `onPrevRoom`, `onNextRoom`, `onRoomAction`, `isActionDisabled`.

`StatsPanel`
Panel se ctyrmi ukazateli stavu mazlicka. Prijima props `stats`. Vnitrne si pro kazdy ukazatel vytvori komponentu `StatChip`.

`StatChip`
Pomocna komponenta uvnitr `StatsPanel`. Zobrazuje nazev statistiky, ciselnou hodnotu a vizualni zaplneni. Prijima props `label`, `value`, `tone`.

`PetCard`
Nejvetsi prezentacni karta herni obrazovky. Zobrazuje aktualni mistnost, penize, samotneho mazlicka, statusovou zpravu a ovladaci tlacitka. Prijima stejne herni props jako `HomeScreen` vcetne `roomIndex` a `roomCount`, ktere pouziva pro plynulou animaci prechodu mezi mistnostmi.

`PetAvatar`
Obaluje obrazek mazlicka a resi fallback, kdyz se obrazek nepovede nacist. Prijima props `src`, `alt`, `children`. V herni obrazovce do sebe vklada komponentu `PetFace`.

`PetFace`
: Kresli oblicej mazlicka pres SVG podle aktualnich statistik. Prijima props `petId` a `stats`. Podle hodnot se meni oci a pusa.

`CutoutImage`
: Pomocna komponenta pro nacteni obrazku a odstraneni svetleho okraje pomoci canvasu. Prijima props `src`, `alt`, `className`, volitelne `onError`.

`StatusModal`
: Modal pro restart, potvrzeni nebo game over. Prijima props `isOpen`, `title`, `message`, `variant`, `onClose`, `onConfirm`.

Tok dat a spoluprace komponent

Zdroj puvodnich dat je v `src/data/pets.ts`, kde jsou definovani tri ukazkovi mazlicci: Bobek, Indicka pneumatika a Bojka. Metadata mistnosti, nazvy, popisy a obrazky jsou v `App.tsx`. Cena a realne dopady akci jsou vyclenene do `src/game/roomActions.ts`, aby sly jednoduse jednotkove testovat. Aplikace pouziva ctyri mistnosti:

- `kitchen` – Kuchyn, akce „Kafe s cigem“, cena 8,
- `store` – Vecerka, akce „Bily monster“, cena 10,
- `park` – Park na hlavnim nadrazi, akce „Dat si piko“, cena 12,
- `casino` – Kasino, akce „Zatocit automatem“, cena 15.

`App` drzi:

- `gameState` – aktualni obrazovku, vybraneho mazlicka a seznam vsech mazlicku,
- `roomIndex` – aktualne vybranou mistnost,
- `coins` – pocet penez,
- `statusText` – posledni informacni zpravu pro hrace,
- `themeMode` a `usesSystemTheme` – svetly/tmavy motiv,
- `modalState` – stav modalniho okna.

Data potom putuji smerem dolu takto:

- `App -> PetSelectionScreen -> PetSelectionList -> PetOptionCard -> PetAvatar -> CutoutImage`
- `App -> HomeScreen -> StatsPanel -> StatChip`
- `App -> HomeScreen -> PetCard -> PetAvatar -> PetFace`
- `App -> PetCard -> CutoutImage` pro mince a ikonu akce
- `App -> StatusModal`

Cista herni pravidla nejsou soucasti prezentacnich komponent. Soubor `src/game/roomActions.ts` obsahuje zejmena `executeRoomAction()`, `clampStat()`, `getMoodFromStats()` a `hasAnyZeroStat()`. Diky tomu lze kontrolovat hranice statu, naladu mazlicka a vysledky akci bez spousteni React rozhrani.

Udalosti se vraceji zpet nahoru pres callbacky:

- klik na `PetOptionCard` zavola `onSelect`, to se prenese az do `App.handleSelectPet()`,
- klik na `SelectPetButton` zavola `App.handleConfirmSelection()`,
- klik na sipky v `PetCard` zavola `App.handlePrevRoom()` nebo `App.handleNextRoom()`,
- klik na hlavni akcni tlacitko v `PetCard` zavola `App.handleRoomAction()`,
- klik na prepinac motivu zavola `App.handleToggleTheme()`,
- klik na novou hru zavola `App.handleRequestRestart()` a nasledne pres modal `App.handleRestart`

5 Sprava stavu

Hlavni stav je spravovan pomoci `useState`, `useMemo`, `useEffect` a `useRef`. Aplikace si uklada rozehranou hru do `localStorage` pod klicem `tamagotchi-wireframe-state`. Motiv se uklada samostatne pod klicem `tamagotchi-theme`.

Ukladany format hry je JSON objekt ve tvaru:

```
{
  "gameState": {
    "currentScreen": "home",
    "selectedPetId": "pet-1",
    "pets": [ ... ]
  },
  "roomIndex": 0,
  "coins": 24,
  "statusText": "Kuchyn je pripravena..."
}
```

Pri nacitani aplikace probehne tento postup:

1. Po mountu `App` zkontroluje, jestli je v `localStorage` ulozeny predchozi stav.
2. Pokud data existuji a maji zakladni spravny tvar, aplikace je nacte do stavu.
3. Pokud jsou data rozbita nebo nevalidni, zobrazi se systemova chyba a uzivatel muze hru restartovat.
4. Po dokonceni hydratace se zmeny stavu automaticky znovu ukladaji.

Pri bezne akci ve hre je tok stavu nasledujici:

1. Uzivatel klikne na akcni tlacitko v aktualni mistnosti.
2. Zavola se `handleRoomAction()`.
3. Funkce zkontroluje, jestli je mazlicek nazivu a jestli ma hrac dostatek penez.
4. Nasledne se pomoci `executeRoomAction()` spocita novy stav statistik podle aktualni mistnosti.
5. `setGameState()` prepise konkretniho mazlicka v poli `pets`.
6. `setCoins()` upravi pocet penez a `setStatusText()` prepise informacni zpravu.

7. React komponenty se prekresli a uzivatel okamzite vidi novy stav.
8. Vedle toho se zmeneny stav automaticky znovu ulozi do `localStorage`.

Casovana herni smycka funguje takto:

- kazdych 12 sekund se snizi `food`, `health`, `happiness` a `energy`,
- kazdych 5 sekund se pricte 1 mince,
- kdyz libovolny stat klesne na 0, otevre se modal „Mazlicek zemrel!!“ a hra vyzaduje restart.

Z aktualnich statistik se zaroven odvozuje nalada mazlicka. Podle nizkeho zdravi, energie nebo hladu se meni `mood` a kresleny vyraz v komponentne `PetFace`. Pokud ma mazlicek malo energie, `PetCard` zobrazi nad avatarem bublinu `zzz`.

6 Animace a vzhled

Rozlozeni aplikace vychazi z mobilniho wireframu. Hra je zabalena do vzhledu telefonni obrazovky, kde je nahore hlavicka, uprostred aktivni obrazovka a dole kratka informacni paticka. Herni cast ma samostatny panel statistik a velkou kartu mistnosti.

Prechazeni mezi mistnostmi neni pouze okamzita zmena obrazku. Komponenta `PetCard` si pamatuje predchozi a aktualni obrazek mistnosti. Pri kliknuti na sipku vytvori dve vrstvy pozadi vedle sebe a pomoci CSS animace je posune. Smer animace se pocita podle `roomIndex` a `roomCount`, aby spravne fungoval i prechod z posledni mistnosti na prvni a z prvni na posledni.

Aplikace ma take svetly a tmavy motiv. Pocitat se umi jak se systemovym nastavenim prohlizece, tak s rucnim prepinacem. Vybrany motiv se uklada do `localStorage`.

7 Nasazeni

Aplikace bezi na GitHub Pages:

https://cqeta1564.github.io/nejlepsi-tamagotchi-na-celym-sigma-svete/

Standardni build prikaz v projektu je:

```
npm run build
```

Pri uspesnem buildu vznika vystup ve slozce `dist/`. Tato slozka obsahuje zejmena:

- `dist/index.html`,
- `dist/assets/index-*.js`,
- `dist/assets/index-*.css`,
- staticke assety z `public/`,
- produkcni `dist/sw.js` vygenerovany Vite pluginem.

Nasazeni je automatizovane pres workflow `.github/workflows/deploy-pages.yml`:

1. po pushi do vetve `main` nebo po rucnim spusteni workflow se spusti GitHub Actions,

2. workflow nastavi Node.js 24,
3. nainstaluje zavislosti prikazem `npm ci`,
4. spusti `npm test`,
5. spusti `npm run lint`,
6. spusti `npm run build`,
7. adresar `dist/` nahraje pomoci `actions/upload-pages-artifact`,
8. nasledne ho pomoci `actions/deploy-pages` publikuje na GitHub Pages.

Soubor `vite.config.ts` nastavuje `base` cestu dynamicky. Lokalne je zaklad `/`, ale v GitHub Actions se cesta odvodi podle nazvu repozitare, aby aplikace spravne fungovala pod GitHub Pages URL.

8 PWA

Aktualni verze projektu ma zapojenou zakladni PWA infrastrukturu. Nejde o rozsahlou offline aplikaci se slozitymi strategiemi cache, ale pro potreby projektu obsahuje manifest, ikony, service worker a registraci service workeru v produkcnim vstupu aplikace.

V repozitari se nachazi:

- `public/manifest.webmanifest` s nazvem, jazykem, ikonami, `display: standalone`, `theme_color` a `background_color`,
- PWA ikony `pwa-192x192.png`, `pwa-512x512.png`, `maskable-icon-512x512.png` a `apple-touch-icon.png`,
- `src/registerServiceWorker.ts`, kde je funkce pro registraci `sw.js`,
- vlastni Vite plugin v `vite.config.ts`, ktery pri produkcnim buildu generuje `dist/sw.js`.

PWA cast je v produkcnim vstupu aplikace zapojena temito vazbami:

- `src/main.tsx` importuje a vola `registerServiceWorker()`,
- `index.html` obsahuje HTML odkaz na `manifest.webmanifest` pres `%BASE_URL%`.

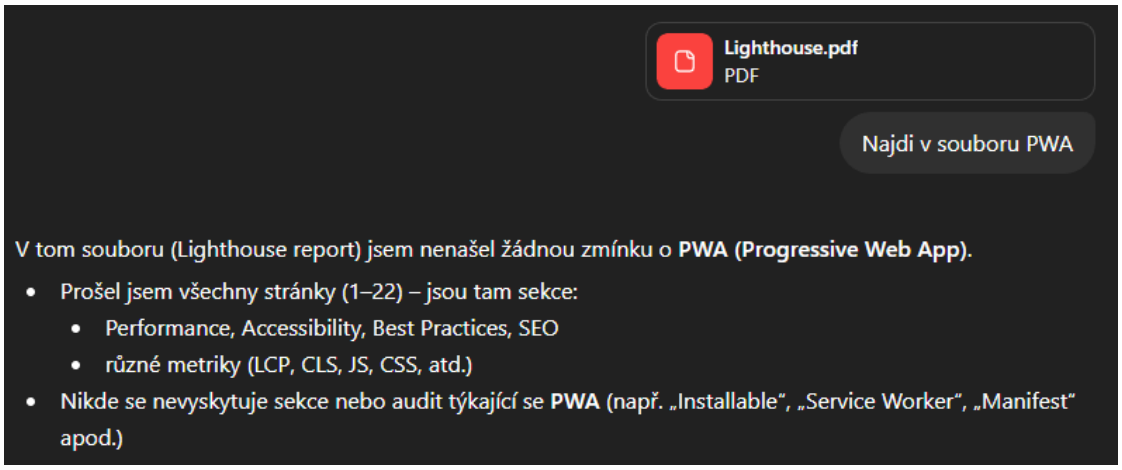
Prakticky to znamena, ze produkcni build pripravuje manifestove assety, generuje service worker a aplikace ho pri produkcnim spusteni registruje. PWA artefakty se overuji prikazem `npm run verify:pwa`, ktery po produkcnim buildu kontroluje manifest, povinne ikony, odkaz v `dist/index.html` a vygenerovany `sw.js` se zakladnimi event listenery.

9 Testovani

Repozitar obsahuje zakladni automaticke testy spustitelne prikazem `npm test`. Testy pouzivaji vestaveny Node.js test runner, samostatny TypeScript build pres `tsconfig.test.json` a pro React smoke testy programove spusteny Vite SSR.

Aktualne automaticke testy pokryvaji cistou herni logiku v `src/game/roomActions.ts`:

- orezani hodnot statu do rozsahu 0–100,



Obrázek 1: Lighthouse kontrola PWA vystupu. Soubor je ulozen jako `docs/lighthouse.png`.

- odvozovani nalady mazlicka podle nizkych statu,
- detekci stavu na nule nebo pod nulou,
- vysledky akci v mistnostech vcetne deterministicke kontroly kasina.

Doplneny jsou take zakladni integracni smoke testy React rozhrani v `tests/react-ui.integration.tes` Ty pres Vite SSR vyrenderuji obrazovku vyberu mazlicka a herni obrazovku, a kontroluji, ze se v HTML objevi hlavni texty, statistiky, penize, vybrany mazlicek a akcni ovladani.

Pri lokalni kontrole 27. 4. 2026 dopadly tyto kontroly takto:

- `npm test` – proslo bez chyb, celkem 8 automatickych testu,
- `npm run lint` – proslo bez chyb,
- `npm run build` – proslo, TypeScript i Vite build do `dist/` byly uspesne,
- `npm run verify:pwa` – proslo, PWA manifest, ikony, HTML odkaz a generovany service worker byly zkontrolovany.

CI workflow na GitHub Actions spousti testy, lint, produkcni build a PWA kontrolu pred nahranim vystupu na GitHub Pages. Automaticke testy pokryvaji herni logiku a zakladni render hlavniho React rozhrani, rucni scenare ale stale overuji chovani v realnem prohlizeci.

Rucne prosle testovaci scenare:

Scenar 1 – prazdny start bez ulozene hry

- Vstupni podminka: v `localStorage` neni zadny klic `tamagotchi-wireframe-state`.
- Kroky: otevru aplikaci poprve.
- Ocekavany vysledek: zobrazi se obrazovka pro vyber mazlicka a tlacitko pro potvrzeni bude na zacatku neaktivni.
- Skutecny vysledek: odpovida ocekavani.

Scenar 2 – pokus pokracovat bez vyberu mazlicka

- Vstupni podminka: aplikace je na obrazovce vyberu, zadny mazlicek neni oznacen.
- Kroky: pokusim se pokracovat bez predchoziho vyberu.
- Ocekavany vysledek: tlacitko „potvrdit vyber“ zustane zablokovane a hra se nespusti.
- Skutecny vysledek: odpovida ocekavani.

Scenar 3 – vyber mazlicka a start hry

- Vstupni podminka: aplikace je na uvodni obrazovce.
- Kroky: vyberu jednoho mazlicka a kliknu na „potvrdit vyber“.
- Ocekavany vysledek: zobrazi se herni obrazovka, aktualni mistnost je kuchyn a hrac ma 24 penez.
- Skutecny vysledek: odpovida ocekavani.

Scenar 4 – provedeni akce v mistnosti

- Vstupni podminka: hra bezi, mazlicek je nazivu a hrac ma dostatek penez.
- Kroky: v kuchyni kliknu na tlacitko akce.
- Ocekavany vysledek: odectou se penize, zmeni se statistiky mazlicka a prepise se statusova zprava.
- Skutecny vysledek: odpovida ocekavani.

Scenar 5 – nedostatek penez

- Vstupni podminka: hra bezi, ale hrac nema dost penez na aktualni akci.
- Kroky: prepnu se do drazsi mistnosti a pokusim se akci spustit.
- Ocekavany vysledek: akcni tlacitko bude neaktivni, aby neslo utratit vic penez, nez je k dispozici.
- Skutecny vysledek: odpovida ocekavani.

Scenar 6 – casovy pokles statu

- Vstupni podminka: hra bezi na domovske obrazovce.
- Kroky: nic neudelam a pockam alespon 12 sekund.
- Ocekavany vysledek: hlad, zdravi, stesti a energie klesnou podle hernich pravidel a zmeni se statusova zprava.
- Skutecny vysledek: odpovida ocekavani.

Scenar 7 – prechod mezi mistnostmi

- Vstupni podminka: hra bezi a je videt hlavni karta mistnosti.
- Kroky: kliknu na levou nebo pravou sipku, pripadne prejdu z posledni mistnosti na prvni.
- Ocekavany vysledek: zmeni se nazev, popis, ikona akce i pozadi mistnosti; pozadi se plynule posune jako sousedni obrazovky.
- Skutecny vysledek: odpovida ocekavani.

Scenar 8 – game over

- Vstupni podminka: nektery stat mazlicka je tesne nad nulou.
- Kroky: necham bezet hru, dokud jeden stat neklesne na 0.
- Ocekavany vysledek: otevre se modal s informaci o smrti mazlicka a hra vyzaduje restart.
- Skutecny vysledek: odpovida ocekavani.

Scenar 9 – obnova po reloadu

- Vstupni podminka: hra je rozehrana a v `localStorage` je ulozen stav.
- Kroky: obnovim stranku v prohlizeci.
- Ocekavany vysledek: aplikace nacte predchoziho mazlicka, mistnost, penize i posledni stav hry.
- Skutecny vysledek: odpovida ocekavani.

Scenar 10 – rozbita ulozena data

- Vstupni podminka: v `localStorage` je schvalne vlozen nevalidni JSON nebo spatny tvar dat.
- Kroky: obnovim stranku.
- Ocekavany vysledek: zobrazi se systemova chyba a tlacitko pro novy zacatek.
- Skutecny vysledek: odpovida ocekavani.

Scenar 11 – svetly a tmavy motiv

- Vstupni podminka: aplikace je otevrena a prohlizec podporuje `prefers-color-scheme`.
- Kroky: kliknu na tlacitko pro prepnuti motivu a potom obnovim stranku.
- Ocekavany vysledek: motiv se prepne a po reloadu zustane ulozeny.
- Skutecny vysledek: odpovida ocekavani.

Scenar 12 – chyba site / offline stav

- Vstupni podminka: uzivatel odpoji internet a chce aplikaci otevrit znovu.
- Kroky: vypnu pripojeni a zkusim obnovit nebo znovu otevrit nasazenou aplikaci.
- Ocekavany vysledek: po prvnim online nacteni by service worker mel umet vratit app shell a prednactene assety.
- Skutecny vysledek: offline provoz je potreba jeste rucne overit v produkcnim prohlizeci.

10 Rozdeleni prace

Z historie vetvi a commitu je videt, ze prace byla delena paralelne mezi wireframe/UI cast a logickou/infrastrukturni cast:

- Martin Veber (Role A – UI): priprava wireframu, barevneho stylu, mobilniho layoutu, pet scen, HUD prvku, animaci mistnosti, tmaveho/svetleho vzhledu a finalniho vizualniho poladeni.
- Matyas Potocký (Role B – logika): priprava datovych typu, vyberu mazlicka, spravy stavu, ukladani do `localStorage`, casove herni smycky, jednotkovych testu, CI, branch protection, nasazeni na GitHub Pages a PWA vystupu.
- Obe role na sebe navazovaly: nejdriv vznikl wireframe a komponentovy zaklad, potom se doplnila herni logika, persistence a deploy, a nakonec se vzhled znovu doladil vcetne animaci, testu, PWA zapojeni a produkcniho buildu.

11 Zhodnoceni projektu

Co se podarilo

- Podarilo se vytvorit funkcni browserovou hru, ktera ma jasny herni cil a jednoduche ovladani.
- Vysledna aplikace pusobi vizualne jednotne a mobile-first rozlozeni je na tak maly projekt nadprumerne propracovane.
- Dobre funguje spojeni logiky a UI: zmeny statu se okamzite projevuji v obraze mazlicka, penizich i informacni hlasce.
- Prakticka je persistence do `localStorage`, diky ktere uzivatel o rozehranou hru neprijde po refreshi.
- Pozitivni je take to, ze projekt ma CI workflow, verejne nasazeni na GitHub Pages a aktualne prochazi testy, lint i produkcnim buildem.
- Povedlo se doplnit plynuly prechod mezi mistnostmi, svetly/tmavy motiv, zaklad PWA, jednotkove testy a React smoke testy.
- Kodova zakladna je uklizenejsi: stare nepouzivane prototypove komponenty byly odstraneny a README odpovida aktualni implementaci.

Co bylo problematicke

- PWA cast je technicky zapojena, ma automatickou kontrolu artefaktu a v priloze je doplneny Lighthouse screenshot.
- Automaticke testy uz kontroluji herni logiku i zakladni render React obrazovek, ale nepokryvaji vsechny klikaci scenare v realnem prohlizeci.
- Pro jeste presnejsi kontrolu by slo doplnit plne prohlizecove end-to-end testy.

Co by slo dale rozvijet

- Dolozit instalaci PWA rucnim testem primo v prohlizeci.
- Rozsirit stavajici automaticke testy o plne prohlizecove end-to-end scenare.
- Rozsirit hru o dalsi mistnosti, vice typu akci, inventar nebo vice druhu mazlicku.
- Pridat zvuky, denni odmeny, skore nebo statistiku delky preziti.
- Pravidelne udrzovat dokumentaci pri dalsich zmenach.

12 Prilohy

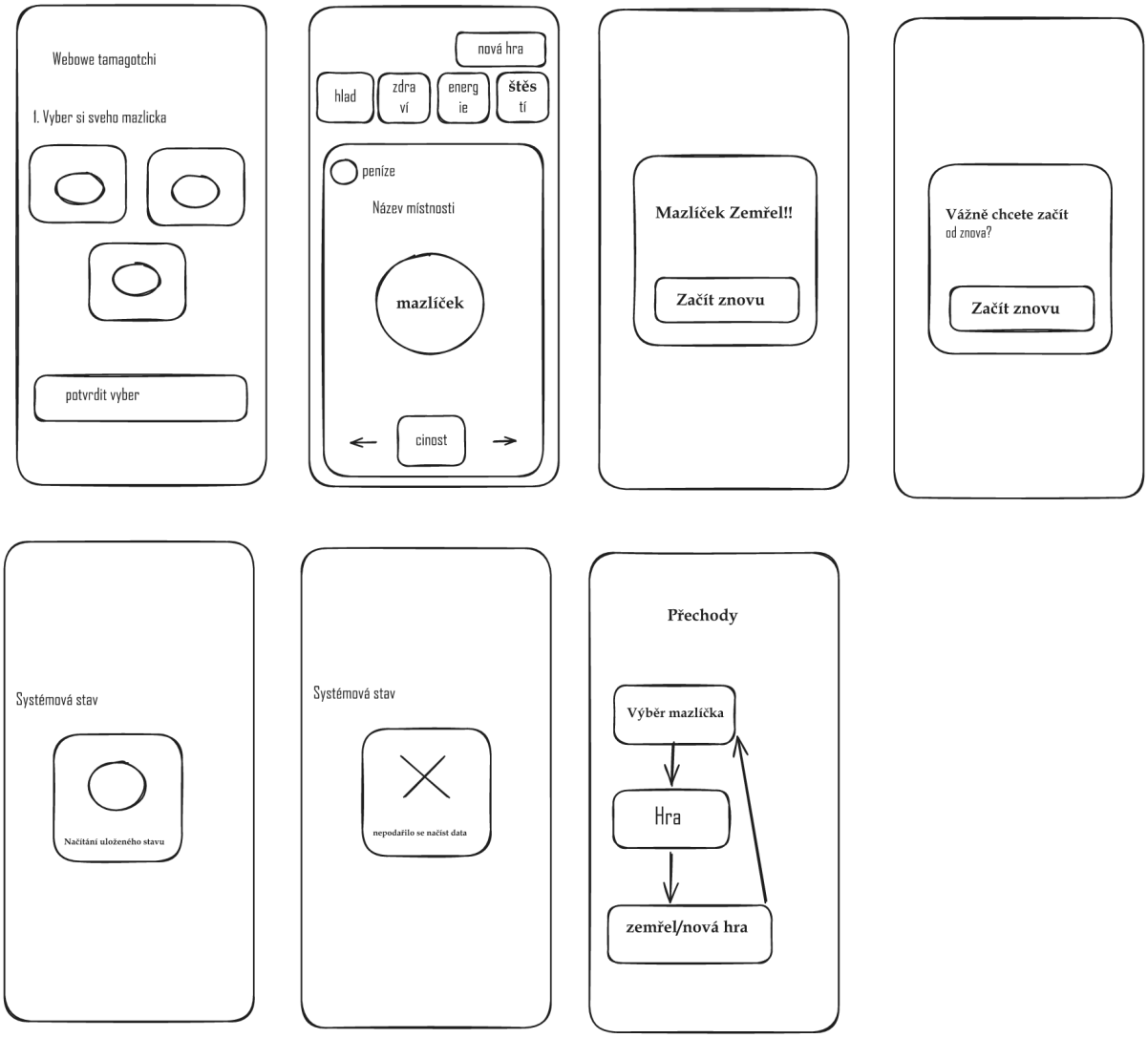
Wireframe z prvniho tydne

Puvodni wireframe je ulozen v repozitari jako `docs/wireframe.svg` a pro vlozeni do zpravy je k dispozici i jako `docs/wireframe.png`.

Screenshoty finalni aplikace

V repozitari jsou ulozene screenshoty finalni aplikace z mobilni i desktopove sirky. Oba soubory jsou ve slozce `docs/` a slouzi jako vizualni doklad aktualniho stavu rozhrani:

- `docs/obrazovkaHryTelefon.png`,
- `docs/obrazovkaHryPocitac.png`.



4 místnosti **Kuchyň, Večerka, Hlávní nádraží park, Kasíno**

Hlad: #FFB347

Zdraví: #4CD97B

Energie: #5AC8FA

Štěstí: #FFD93D

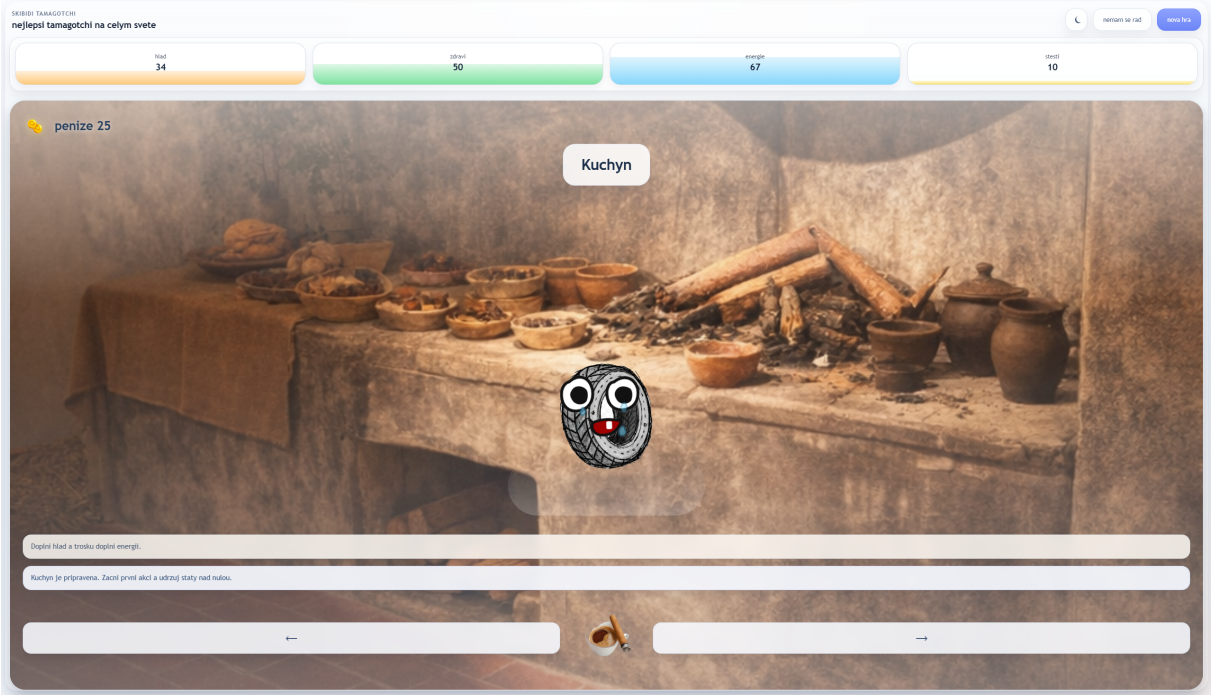
#aaa9b4

#000000

#9e9bbb

Obrázek 2: Puvodni wireframe aplikace z prvniho tydne vyvoje. Soubor je ulozen jako `docs/wireframe.png`.

Obrázek 3: Finalni aplikace v mobilni sirce. Soubor je ulozen jako `docs/obrazovkaHryTelefon.png`.



Obrázek 4: Finalni aplikace v desktopove sirce. Soubor je ulozen jako `docs/obrazovkaHryPocitac.png`.